

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине
Вычислительная математика
Лабораторная работа №3
«Слайны»
Вариант 18

Выполнил:
студент группы 5130201/40003
Джабраилов А.В.
Подпись: _____

Принял:
К. физ.-мат. наук, доц., доц.
Пак В.Г.
Подпись: _____
«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Задание

1. Построить по таблице интерполяционный кубический сплайн с заданными граничными условиями.
2. Построить по таблице интерполяционный квадратичный сплайн с заданными точками соединения и граничными условиями.

2 Цель работы

Освоить алгоритмы построения кубических и квадратичных интерполяционных сплайнов, реализовать их численно, проверить условия интерполяции и гладкости.

3 Ход работы

Таблица 1: Таблица исходных данных.

i	x_i	y_i
0	-1,000	1,3680
1	-0,010	1,9900
2	0,750	3,1170
3	1,900	7,6860
4	2,815	17,6920
5	3,100	23,1980
6	3,419	31,5390
7	3,875	49,1830
8	4,200	67,6860

Алгоритм вычислений:

1. **Кубический сплайн:** Формирование СЛАУ для наклонов $m_i = S'(x_i)$:
$$\mu_i m_{i-1} + 2m_i + \lambda_i m_{i+1} = d_i.$$

Добавление естественных граничных условий: $2m_0 + m_1 = 3 \frac{y_1 - y_0}{h_0}$,
 $m_{n-1} + 2m_n = 3 \frac{y_n - y_{n-1}}{h_{n-1}}.$

Решение системы, вычисление значений сплайна на каждом отрезке $[x_i, x_{i+1}]$.

2. **Квадратичный сплайн:** Введение узлов $\tilde{x}_i$.

Построение $3(n + 1)$ СЛАУ для коэффициентов a_i, b_i, c_i полиномов $S_i(x) = a_i x^2 + b_i x + c_i$.

из условий:

- интерполяции в узлах x_i ;
- непрерывности $S_{i-1}(\tilde{x}_i) = S_i(\tilde{x}_i)$;
- гладкости $S'_{i-1}(\tilde{x}_i) = S'_i(\tilde{x}_i)$;
- граничных условий $S'_0(x_0) = 0, S'_n(x_n) = 0$.

Решение СЛАУ.

Для реализации использован язык *Python* с библиотекой *numpy* для решения СЛАУ. Исходный код приведён в приложении.

4 Результаты

4.1 Кубический сплайн (естественные граничные условия)

Вычисленные наклоны $m_i = S'(x_i)$ в узлах:

Таблица 2: Наклоны кубического сплайна в узлах.

i	m_i
0	0,507947
1	0,868954
2	2,240519
3	6,842447
4	14,086445
5	22,693660
6	31,700381
7	47,324849
8	61,736037

Значения кубического сплайна $S(x)$:

Таблица 3: Значения кубического сплайна.

x	$S(x)$	x	$S(x)$
-1,0000	1,368000	0,0947	2,088085
-0,7263	1,509534	0,3684	2,420919
-0,4526	1,666170	0,6421	2,888753
-0,1789	1,853009	0,9158	3,506608
x	$S(x)$	x	$S(x)$
1,1895	4,274650	2,5579	14,248997
1,4632	5,290071	2,8316	17,932273
1,7368	6,658605	3,1053	23,317561
2,0105	8,507976	3,3789	30,306474
x	$S(x)$	x	$S(x)$
2,2842	11,063452	3,6526	39,686545
3,9263	51,722133	4,2000	67,686000

Проверка интерполяции в исходных узлах:

4.2 Квадратичный сплайн (точки соединения – середины отрезков)

Коэффициенты квадратичных полиномов $S_i(x) = a_i x^2 + b_i x + c_i$:

Таблица 4: Проверка кубического сплайна.

x_i	y_i	$S(x_i)$	$ y_i - S(x_i) $
-1,0000	1,368000	1,368000	$0,00 \cdot 10^0$
-0,0100	1,990000	1,990000	$0,00 \cdot 10^0$
0,7500	3,117000	3,117000	$0,00 \cdot 10^0$
1,9000	7,686000	7,686000	$0,00 \cdot 10^0$
2,8150	17,692000	17,692000	$0,00 \cdot 10^0$
3,1000	23,198000	23,198000	$0,00 \cdot 10^0$
3,4190	31,539000	31,539000	$0,00 \cdot 10^0$
3,8750	49,183000	49,183000	$0,00 \cdot 10^0$
4,2000	67,686000	67,686000	$0,00 \cdot 10^0$

Таблица 5: Коэффициенты квадратичного сплайна.

i	a_i	b_i	c_i
0	0,725365	1,450730	2,093365
1	0,362422	1,084157	2,000805
2	1,049503	0,575717	2,094867
3	3,192608	-5,103511	5,857355
4	7,502476	-25,424536	29,810763
5	12,649009	-55,866281	74,826494
6	7,065638	-19,468286	15,506861
7	65,175319	-443,320300	788,401009
8	-255,293548	2144,465805	-4435,692191

Значения квадратичного сплайна $S(x)$:

Проверка интерполяции в исходных узлах:

Таблица 6: Значения квадратичного сплайна.

x	$S(x)$	x	$S(x)$
-1,0000	1,368000	0,0947	2,106768
-0,7263	1,422332	0,3684	2,449425
-0,4526	1,584333	0,6421	2,897247
-0,1789	1,818404	0,9158	3,502290
x	$S(x)$	x	$S(x)$
1,1895	4,264554	2,5579	13,864866
1,4632	5,224947	2,8316	17,972827
1,7368	6,624250	3,1053	23,317074
2,0105	8,501824	3,3789	30,394955
x	$S(x)$	x	$S(x)$
2,2842	10,857671	3,6526	38,665976
3,9263	52,525349	4,2000	67,686000

Таблица 7: Проверка квадратичного сплайна.

x_i	y_i	$S(x_i)$	$ y_i - S(x_i) $
-1,0000	1,368000	1,368000	$0,00 \cdot 10^0$
-0,0100	1,990000	1,990000	$0,00 \cdot 10^0$
0,7500	3,117000	3,117000	$8,88 \cdot 10^{-16}$
1,9000	7,686000	7,686000	$6,22 \cdot 10^{-15}$
2,8150	17,692000	17,692000	$2,49 \cdot 10^{-14}$
3,1000	23,198000	23,198000	$3,06 \cdot 10^{-13}$
3,4190	31,539000	31,539000	$1,38 \cdot 10^{-12}$
3,8750	49,183000	49,183000	$8,03 \cdot 10^{-13}$
4,2000	67,686000	67,686000	$3,13 \cdot 10^{-13}$

Как видно из правого столбца, есть отклонения, но они пренебрежительно малы и обоснованы машинной точностью.

5 Приложение

Listing 1: Файл lab3.py

```

1 import numpy as np
2
3
4 # =====
5 # Data
6 # =====
7 x_data = np.array([-1.000, -0.010, 0.750, 1.900, 2.815, 3.100,
                     3.419, 3.875, 4.200])

```

```

8 y_data = np.array([1.3680, 1.9900, 3.1170, 7.6860, 17.6920,
9 23.1980, 31.5390, 49.1830, 67.6860])
10 n = len(x_data) - 1 # number of intervals
11
12 # =====
13 # 1. Cubic Spline (Natural / Free-end conditions)
14 # =====
15
16 def build_cubic_spline_natural(x, y):
17     """
18     Build natural cubic spline ( $S''(x_0) = S''(x_n) = 0$ ).
19     Returns the slopes  $m_i$  at each node.
20     """
21     n = len(x) - 1
22     h = np.diff(x) #  $h[i] = x[i+1] - x[i]$ 
23
24     # Build tridiagonal system  $A * m = d$ 
25     # Size: (n+1) x (n+1)
26     A = np.zeros((n + 1, n + 1))
27     d = np.zeros(n + 1)
28
29     # Interior equations:  $\mu_i * m_{i-1} + 2*m_i + \lambda_i * m_{i+1} = d_i$ 
30     for i in range(1, n):
31         mu_i = h[i - 1] / (h[i - 1] + h[i])
32         lambda_i = h[i] / (h[i - 1] + h[i])
33         d_i = 3 * (mu_i * (y[i] - y[i - 1]) / h[i - 1] + lambda_i
34 * (y[i + 1] - y[i]) / h[i])
35
36         A[i, i - 1] = mu_i
37         A[i, i] = 2.0
38         A[i, i + 1] = lambda_i
39         d[i] = d_i
40
41     # Natural boundary conditions:  $S''(x_0) = 0, S''(x_n) = 0$ 
42     #  $2*m_0 + m_1 = 3*(y_1 - y_0)/h_0$ 
43     #  $m_{n-1} + 2*m_n = 3*(y_n - y_{n-1})/h_{n-1}$ 
44     A[0, 0] = 2.0
45     A[0, 1] = 1.0
46     d[0] = 3.0 * (y[1] - y[0]) / h[0]

```

```

46
47     A[n, n - 1] = 1.0
48     A[n, n] = 2.0
49     d[n] = 3.0 * (y[n] - y[n - 1]) / h[n - 1]
50
51     m = np.linalg.solve(A, d)
52     return m
53
54
55 def eval_cubic_spline(x, y, m, x_eval):
56     """
57     Evaluate cubic spline at given points using Hermite
58     interpolation formula.
59     """
60     n = len(x) - 1
61     result = np.zeros_like(x_eval, dtype=float)
62
63     for j, xv in enumerate(x_eval):
64         # Find the interval [x_i, x_{i+1}] containing xv
65         if xv <= x[0]:
66             i = 0
67         elif xv >= x[-1]:
68             i = n - 1
69         else:
70             i = np.searchsorted(x, xv) - 1
71             if i < 0:
72                 i = 0
73             if i >= n:
74                 i = n - 1
75
76         h_i = x[i + 1] - x[i]
77         t = (xv - x[i]) / h_i
78
79         # Hermite basis functions
80         phi1 = (1 - t) ** 2 * (1 + 2 * t)
81         phi2 = t ** 2 * (3 - 2 * t)
82         psi1 = t * (1 - t) ** 2
83         psi2 = t ** 2 * (t - 1)
84
85         result[j] = y[i] * phi1 + y[i + 1] * phi2 + h_i * (m[i] *
86         psi1 + m[i + 1] * psi2)

```



```

85
86     return result
87
88
89 # =====
90 # 2. Quadratic Spline (with midpoints as join points , natural
    conditions)
91 # =====
92
93 def build_quadratic_spline(x, y):
94     """
95     Build quadratic spline with join points at midpoints.
96     Each quadratic:  $S_i(t) = a_i * t^2 + b_i * t + c_i$  on [
    xtilde_{i-1}, xtilde_i]
97
98     Join points:  $xtilde_i = (x[i] + x[i+1]) / 2$ 
99     Boundary conditions:  $S'_0(x_0) = 0$ ,  $S'_n(x_n) = 0$  (natural-like
    )
100
101     Returns list of (a_i, b_i, c_i, left_i, right_i) for each
    segment.
102     """
103     n = len(x) - 1
104
105     # Join points (midpoints)
106     xtilde = np.array([(x[i] + x[i + 1]) / 2 for i in range(n)])
107
108     # Extended intervals: [x0, xtilde0], [xtilde0, xtilde1], ...,
    [xtilde_{n-1}, xn]
109     # Total: n+1 quadratic polynomials
110     left_bounds = np.concatenate([[x[0]], xtilde])
111     right_bounds = np.concatenate([xtilde, [x[-1]]])
112
113     # Each quadratic:  $S_i(t) = a_i * t^2 + b_i * t + c_i$ 
114     # Unknowns: 3*(n+1) coefficients
115     num_segments = n + 1
116     num_unknowns = 3 * num_segments
117
118     A = np.zeros((num_unknowns, num_unknowns))
119     b_vec = np.zeros(num_unknowns)
120

```

```

121 row = 0
122
123 # 1. Interpolation conditions:  $S_i(x_i) = y_i$ 
124 for i in range(num_segments):
125     xi = x[i]
126     #  $S_i(xi) = a_i*xi^2 + b_i*xi + c_i = y_i$ 
127     A[row, 3 * i] = xi ** 2
128     A[row, 3 * i + 1] = xi
129     A[row, 3 * i + 2] = 1.0
130     b_vec[row] = y[i]
131     row += 1
132
133 # 2. Continuity at join points:  $S_{i-1}(x_{tilde_i}) = S_i(x_{tilde_i})$ 
134 for i in range(1, num_segments):
135     xt = xtilde[i - 1] # join point between segment i-1 and
136     i
137     #  $S_{i-1}(xt) - S_i(xt) = 0$ 
138     A[row, 3 * (i - 1)] = xt ** 2
139     A[row, 3 * (i - 1) + 1] = xt
140     A[row, 3 * (i - 1) + 2] = 1.0
141     A[row, 3 * i] = -(xt ** 2)
142     A[row, 3 * i + 1] = -xt
143     A[row, 3 * i + 2] = -1.0
144     b_vec[row] = 0.0
145     row += 1
146
147 # 3. Smoothness (C1 continuity) at join points:  $S'_{i-1}(x_{tilde_i}) = S'_i(x_{tilde_i})$ 
148 for i in range(1, num_segments):
149     xt = xtilde[i - 1]
150     #  $S'_{i-1}(xt) - S'_i(xt) = 0$ 
151     #  $S'_i(t) = 2*a_i*t + b_i$ 
152     A[row, 3 * (i - 1)] = 2 * xt
153     A[row, 3 * (i - 1) + 1] = 1.0
154     A[row, 3 * i] = -2 * xt
155     A[row, 3 * i + 1] = -1.0
156     b_vec[row] = 0.0
157     row += 1

```

```

158     # 4. Boundary conditions (natural-like):  $S'_0(x_0) = 0$ ,  $S'_n(x_n) = 0$ 
159     #  $S'_0(x_0) = 2*a_0*x_0 + b_0 = 0$ 
160     A[row, 0] = 2 * x[0]
161     A[row, 1] = 1.0
162     b_vec[row] = 0.0
163     row += 1
164
165     #  $S'_n(x_n) = 2*a_n*x_n + b_n = 0$ 
166     A[row, 3 * n] = 2 * x[-1]
167     A[row, 3 * n + 1] = 1.0
168     b_vec[row] = 0.0
169     row += 1
170
171     # Solve the system
172     coeffs = np.linalg.solve(A, b_vec)
173
174     # Extract polynomials
175     segments = []
176     for i in range(num_segments):
177         a_i = coeffs[3 * i]
178         b_i = coeffs[3 * i + 1]
179         c_i = coeffs[3 * i + 2]
180         segments.append((a_i, b_i, c_i, left_bounds[i],
181 right_bounds[i]))
182
183     return segments
184
185 def eval_quadratic_spline(segments, x_eval):
186     """
187     Evaluate quadratic spline at given points.
188     """
189     result = np.zeros_like(x_eval, dtype=float)
190
191     for j, xv in enumerate(x_eval):
192         for a_i, b_i, c_i, left, right in segments:
193             if left <= xv <= right:
194                 result[j] = a_i * xv ** 2 + b_i * xv + c_i
195                 break
196         else:

```

```

197         # Handle edge cases with small tolerance
198         for a_i, b_i, c_i, left, right in segments:
199             if abs(xv - left) < 1e-10 or abs(xv - right) < 1e
-10:
200                 result[j] = a_i * xv ** 2 + b_i * xv + c_i
201                 break
202
203     return result
204
205
206 # ====
207 # Main
208 # =====
209
210 if __name__ == "__main__":
211     print("=" * 70)
212     print("LAB 3: SPLINE INTERPOLATION")
213     print("=" * 70)
214     print("\nData points:")
215     print(f"    xi = {x_data}")
216     print(f"    yi = {y_data}")
217     print(f"    Number of intervals: {n}")
218
219     # -----
220     # Part 1: Cubic Spline (Natural)
221     # -----
222     print("\n" + "-" * 70)
223     print("PART 1: CUBIC SPLINE (Natural / Free-end)")
224     print("-" * 70)
225
226     m = build_cubic_spline_natural(x_data, y_data)
227     print("\nComputed slopes (m_i) at nodes:")
228     for i, mi in enumerate(m):
229         print(f"    m[{i}] = {mi:.6f}")
230
231     # Evaluate at various points
232     x_eval = np.linspace(x_data[0], x_data[-1], 20)
233     y_eval = eval_cubic_spline(x_data, y_data, m, x_eval)
234
235     print("\nCubic spline evaluation:")
236     print(f"    {'x':>10}    {'S(x)':>12}")

```

```

237     for xv, yv in zip(x_eval, y_eval):
238         print(f"    {xv:10.4f} {yv:12.6f}")
239
240     # Verification: check interpolation at original nodes
241     y_check = eval_cubic_spline(x_data, y_data, m, x_data)
242     print("\nVerification (interpolation at original nodes):")
243     print(f"    {'x':>10} {'y':>12} {'S(x)':>12} {'Error':>12}")
244     for xv, yv, sv in zip(x_data, y_data, y_check):
245         print(f"    {xv:10.4f} {yv:12.6f} {sv:12.6f} {abs(yv - sv)
:12.2e}")
246
247     # -----
248     # Part 2: Quadratic Spline
249     # -----
250     print("\n" + "-" * 70)
251     print("PART 2: QUADRATIC SPLINE (midpoint join points)")
252     print("-" * 70)
253
254     segments = build_quadratic_spline(x_data, y_data)
255
256     print("\nQuadratic polynomials  $S_i(x) = ax^2 + bx + c$ :")
257     for i, (a_i, b_i, c_i, left, right) in enumerate(segments):
258         print(f"    Segment {i}: [{left:.4f}, {right:.4f}]")
259         print(f"        a = {a_i:.6f}, b = {b_i:.6f}, c = {c_i:.6f}")
260
261     # Evaluate at various points
262     y_quad_eval = eval_quadratic_spline(segments, x_eval)
263
264     print("\nQuadratic spline evaluation:")
265     print(f"    {'x':>10} {'S(x)':>12}")
266     for xv, yv in zip(x_eval, y_quad_eval):
267         print(f"    {xv:10.4f} {yv:12.6f}")
268
269     # Verification: check interpolation at original nodes
270     y_quad_check = eval_quadratic_spline(segments, x_data)
271     print("\nVerification (interpolation at original nodes):")
272     print(f"    {'x':>10} {'y':>12} {'S(x)':>12} {'Error':>12}")
273     for xv, yv, sv in zip(x_data, y_data, y_quad_check):
274         print(f"    {xv:10.4f} {yv:12.6f} {sv:12.6f} {abs(yv - sv)
:12.2e}")
275

```

```
276     print("\n" + "=" * 70)
277     print("DONE")
278     print("=" * 70)
```